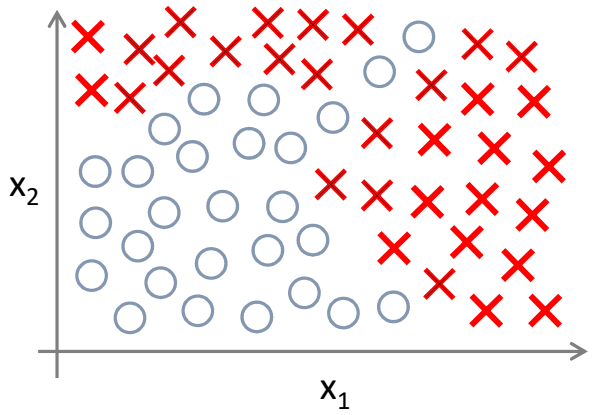
Neural networks

Motivation

Non-linear Classification



$x_1 =$ size
$x_2 =$ # bedrooms
$x_3 =$ # floors
$x_4 =$ age
$\dots$
x_{100}

$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^3 x_2 + \theta_6 x_1 x_2^2 + \dots)$$

$$O(n^2) = 5000$$

Classification problem

You see this:



50 x 50 = 2500
pixel

2500 x 3 = 7500
RGB

$n_i \times n_j$

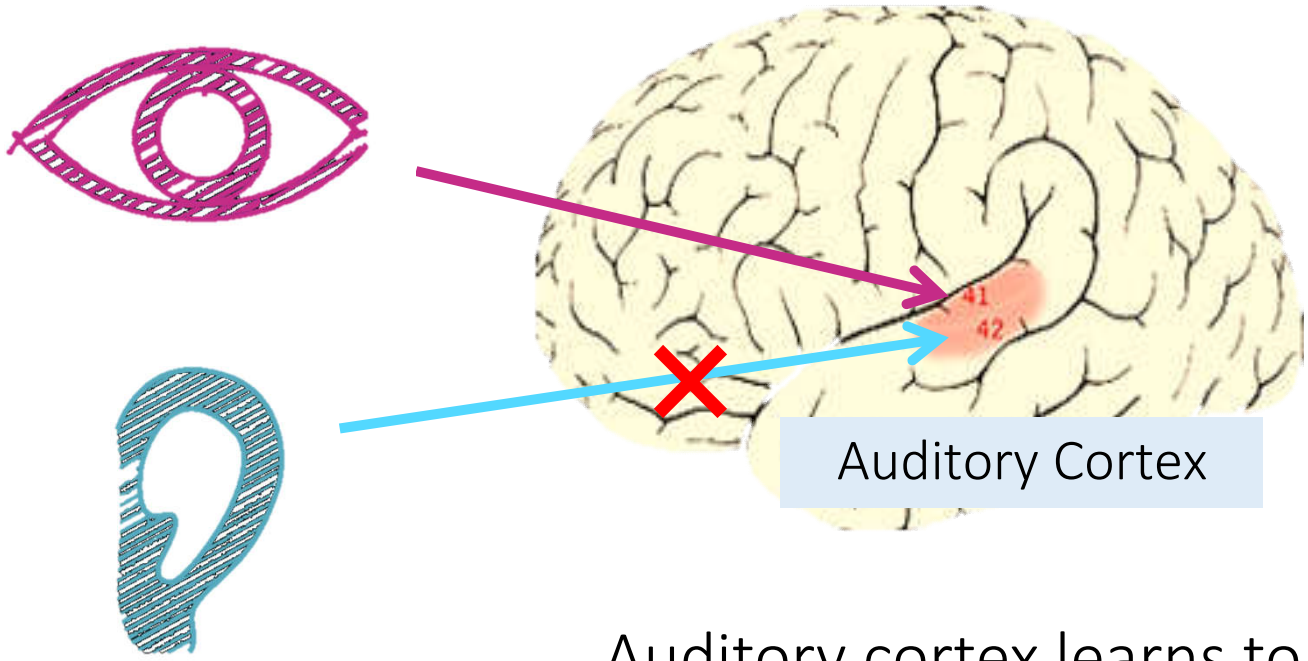
↓

~2,43M

What is our problem?

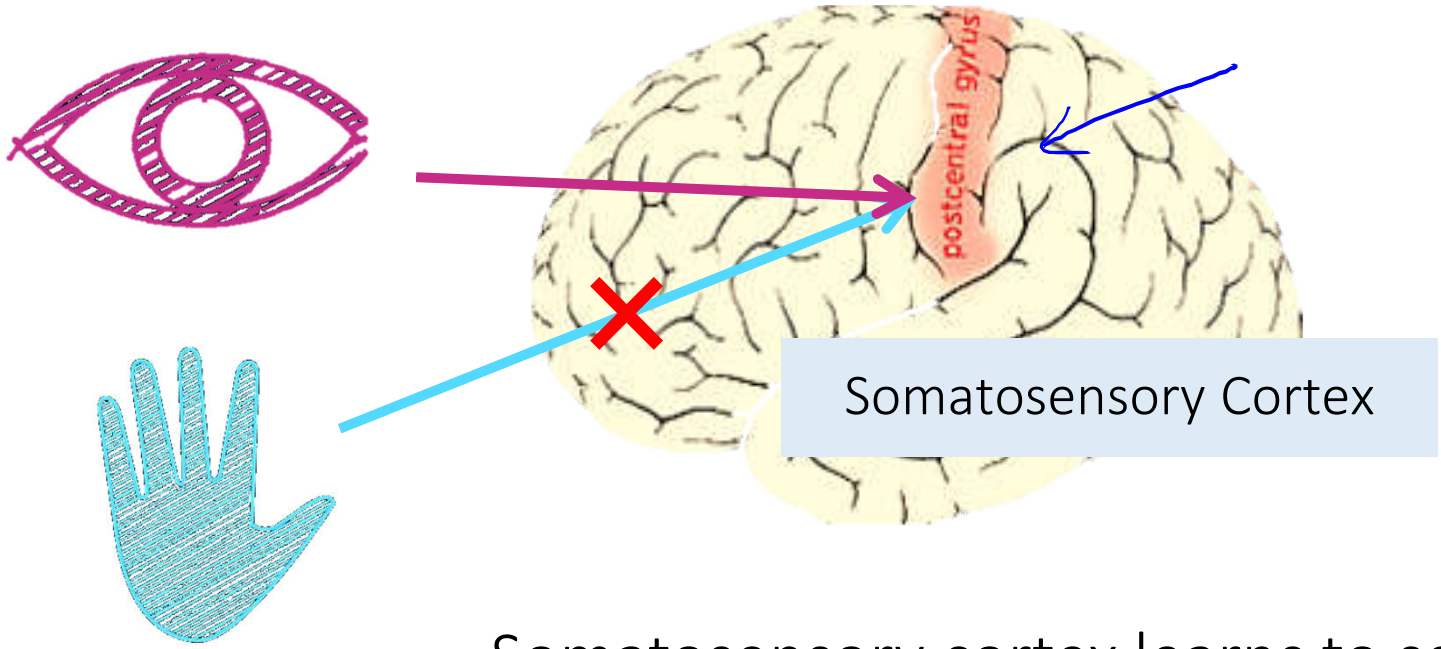
1. Nonlinear
2. Complex models

The “one learning algorithm” hypothesis



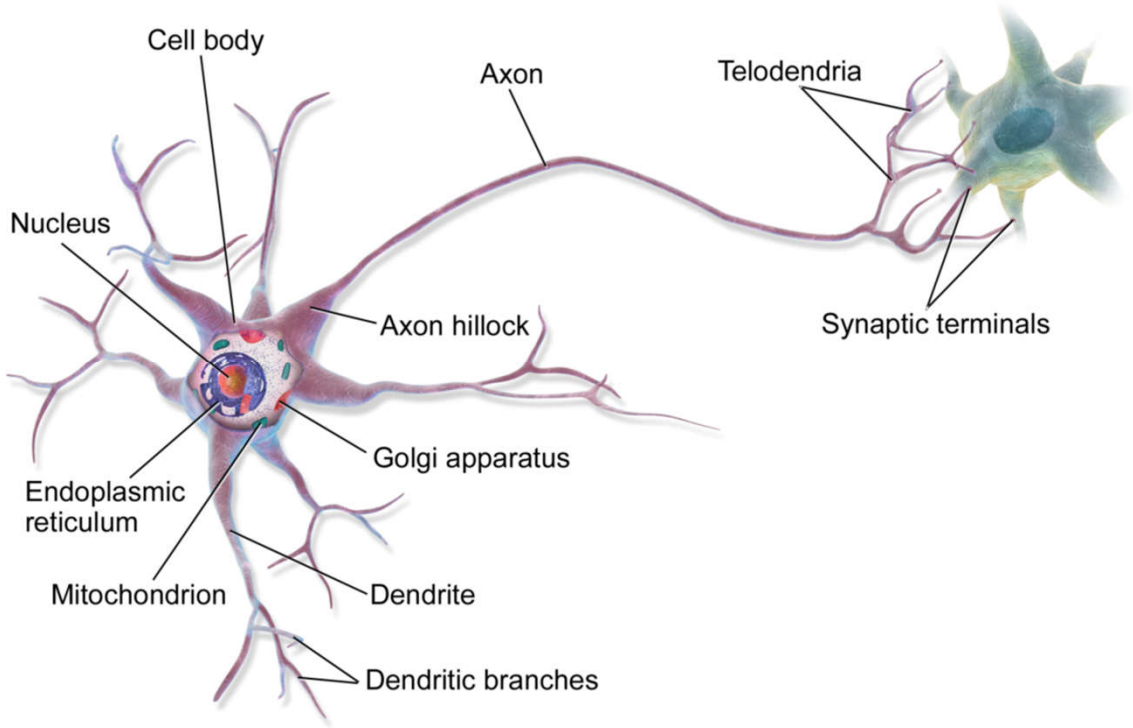
Auditory cortex learns to see

The “one learning algorithm” hypothesis



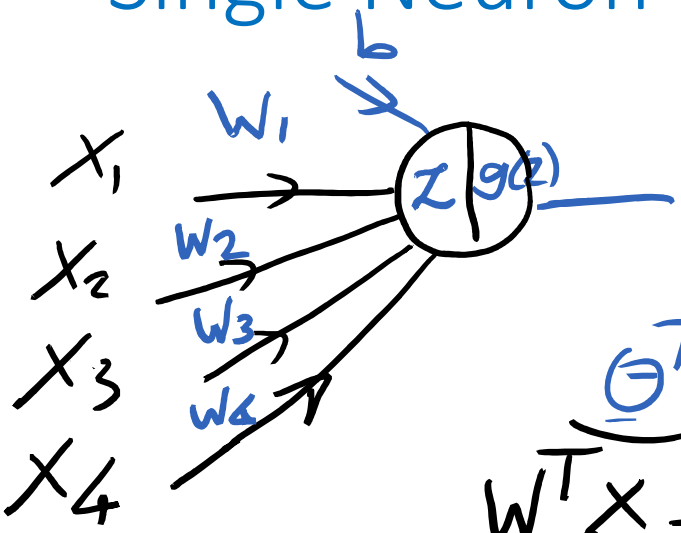
Somatosensory cortex learns to see

Neuron


Cell body
Axon
Telodendria
Nucleus
Axon hillock
Synaptic terminals
Golgi apparatus
Endoplasmic reticulum
Mitochondrion
Dendrite
Dendritic branches

Single Neuron neural network

بیس سی صمت :



$h_\theta(x) = \max(\theta^T X, 0)$

$\theta^T X = \theta_0 X_0 + \theta_1 X_1 + \theta_2 X_2 + \theta_3 X_3 + \theta_4 X_4$

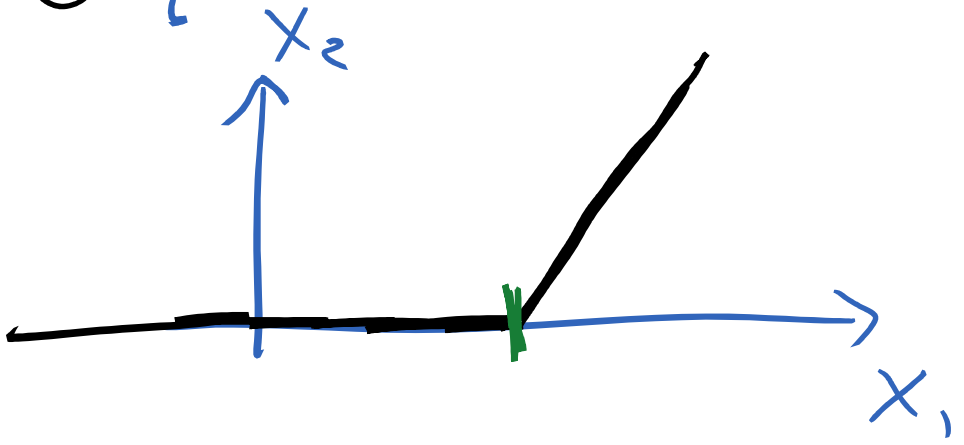
$W^T X + b$

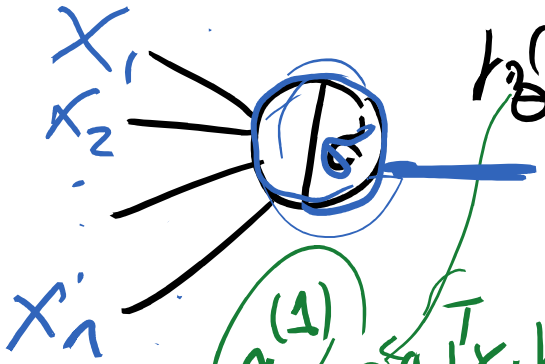
وزن‌ها — بایاس

تابع خطی

$g(z) = \max(0, z)$

ReLU
(Rectified Linear Unit)





$h_\theta(x) = \sigma(w^T x + b)$

طبقه‌بندی باینری

log. با مدل

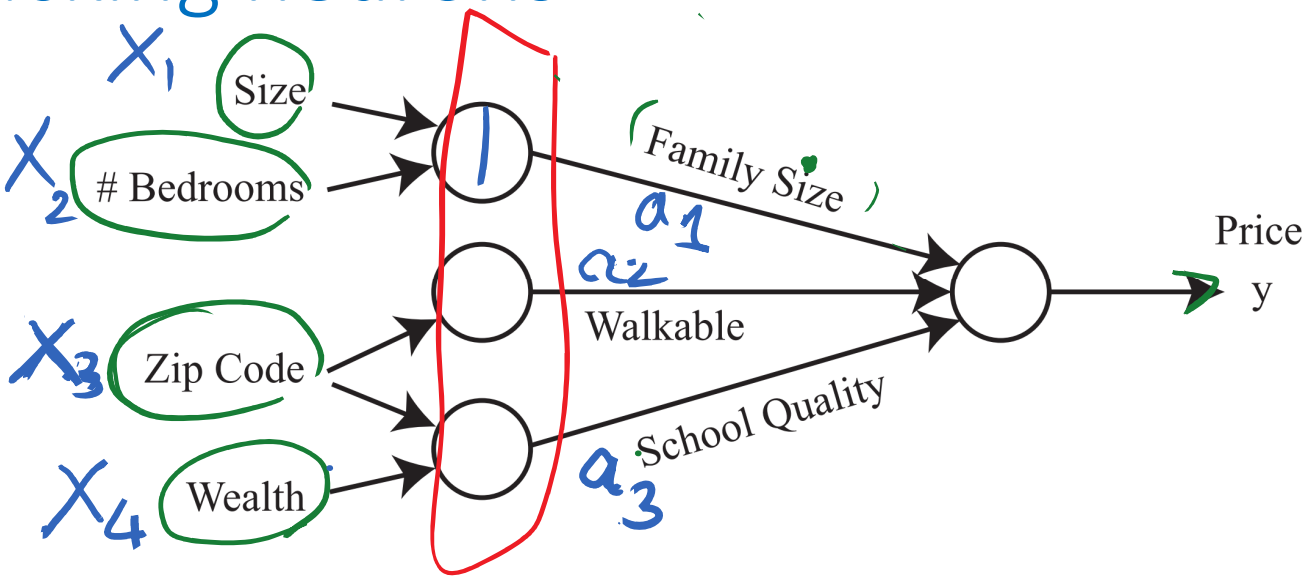
Reg.

$a^{(1)} = g(w_1^T x + b_1)$

$a^{(2)} = g(w_2 a^{(1)} + b_2)$

$$g(z) = \frac{1}{e^{-z} + 1}$$

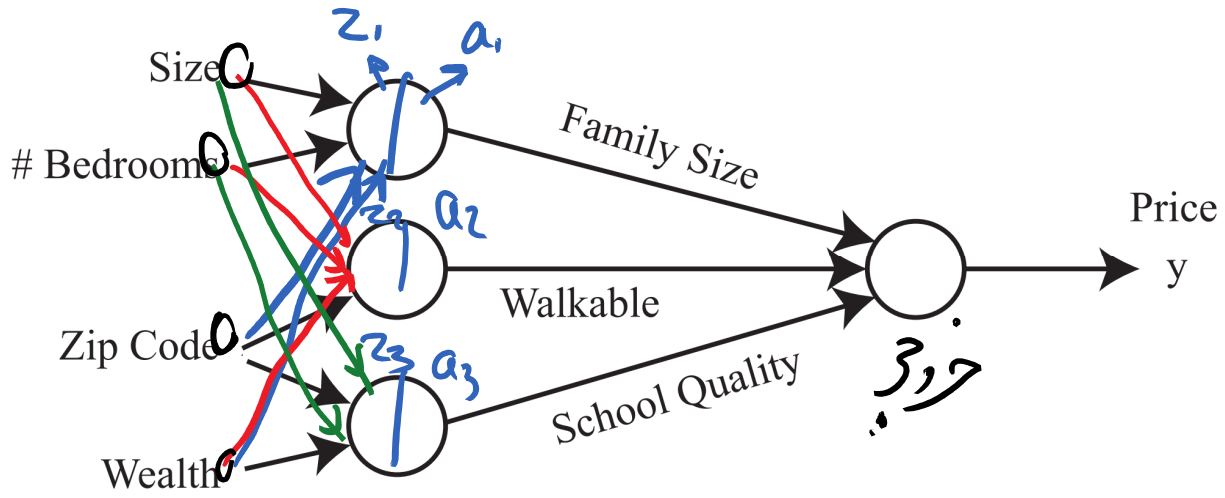
Stacking neurons



$$a_1 = \text{ReLU}(\theta_1 x_1 + \theta_2 x_2 + \theta_3)$$
$$a_2 = \text{ReLU}(\theta_4 x_3 + \theta_5)$$
$$a_3 = \text{ReLU}(\theta_6 x_3 + \theta_7 x_4 + \theta_8)$$

$$\bar{h}_\theta(x) = \theta_9 a_1 + \theta_{10} a_2 + \theta_{11} a_3 + \theta_{12}$$

Two-layer fully connected neural network



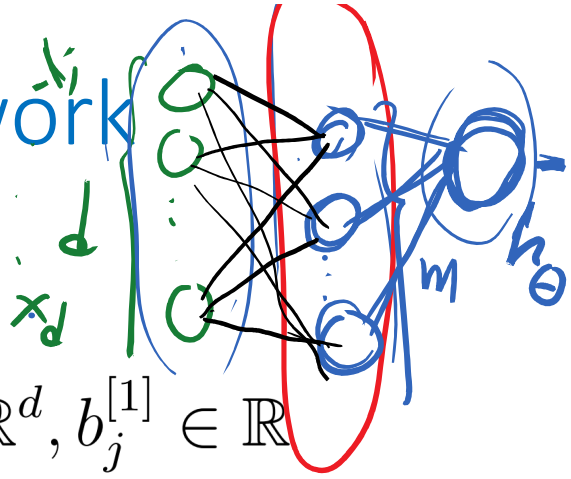
$$a_1 = \text{ReLU}(w_1^\top x + b_1), \text{ where } w_1 \in \mathbb{R}^4 \text{ and } b_1 \in \mathbb{R}$$

$$a_2 = \text{ReLU}(w_2^\top x + b_2), \text{ where } w_2 \in \mathbb{R}^4 \text{ and } b_2 \in \mathbb{R}$$

$$a_3 = \text{ReLU}(w_3^\top x + b_3), \text{ where } w_3 \in \mathbb{R}^4 \text{ and } b_3 \in \mathbb{R}$$

$$h_\theta(x) = \theta_1 a_1 + \theta_2 a_2 + \theta_3 a_3 + \theta_4$$

Two-layer fully connected neural network



$$\forall j \in [1, ..., m], \quad z_j = w_j^{[1]\top} x + b_j^{[1]} \text{ where } w_j^{[1]} \in \mathbb{R}^d, b_j^{[1]} \in \mathbb{R}$$

$$a_j = \text{ReLU}(z_j),$$

$$a = [a_1, \ldots, a_m]^\top \in \mathbb{R}^m$$

$$\bar{h}_\theta(x) = w^{[2]\top} a + b^{[2]} \text{ where } w^{[2]} \in \mathbb{R}^m, b^{[2]} \in \mathbb{R},$$

$$\underline{W}^{[1]} = \begin{bmatrix} w_1^{[1]\top} \\ w_2^{[1]\top} \\ \vdots \\ w_m^{[1]\top} \end{bmatrix}_{m \times d} \qquad z_{m\times 1} = \begin{bmatrix} z_1 \\ \vdots \\ z_m \end{bmatrix}$$

$$\underset{m\times 1}{z} = \underset{m\times d}{W^{[1]}} \underset{d\times 1}{X} + \underset{m\times 1}{b^{[1]}} \qquad \underset{m\times 1}{a} = \text{ReLU}(z)$$

ماتریسی-برداری کردن ← GPU

برای محاسبات سریع‌تر

$$MM \rightarrow g \rightarrow MM \rightarrow g \cdots \rightarrow g \rightarrow h_\theta(x)$$

$$a^{(1)} = g(W^{[1]T} x + b) \qquad a^{(2)} = g(W^{[2]T} a + b)$$

$\theta^T x$

تابع فعال‌ساز (غیرخطی)

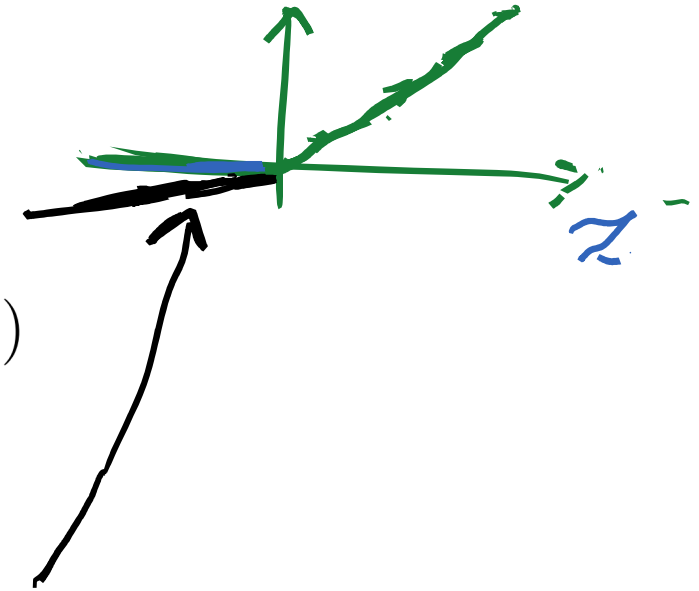
$$h_\theta(x) = f_l(f_{l-1}(f_{l-2}(\cdots(f_1(x)))$$

Multilayer fully connected neural network

$$
\begin{aligned}
a^{[1]} &= \mathrm{ReLU}(W^{[1]}x + b^{[1]}) \\
a^{[2]} &= \mathrm{ReLU}(W^{[2]}a^{[1]} + b^{[2]}) \\
&\dots \\
a^{[r-1]} &= \mathrm{ReLU}(W^{[r-1]}a^{[r-2]} + b^{[r-1]}) \\
\bar{h}_\theta(x) &= W^{[r]}a^{[r-1]} + b^{[r]}
\end{aligned}
$$

نقشه‌ی ویژگی توسط شبکه یاد گرفته شده ∅ (X)

Activation Functions



$$\sigma(z) = \frac{1}{1+e^{-z}} \qquad \text{(sigmoid)}$$

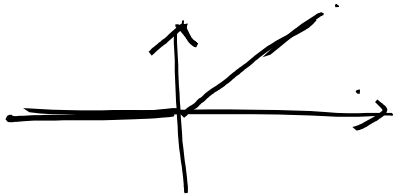
$$\sigma(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}} \qquad \text{(tanh)}$$

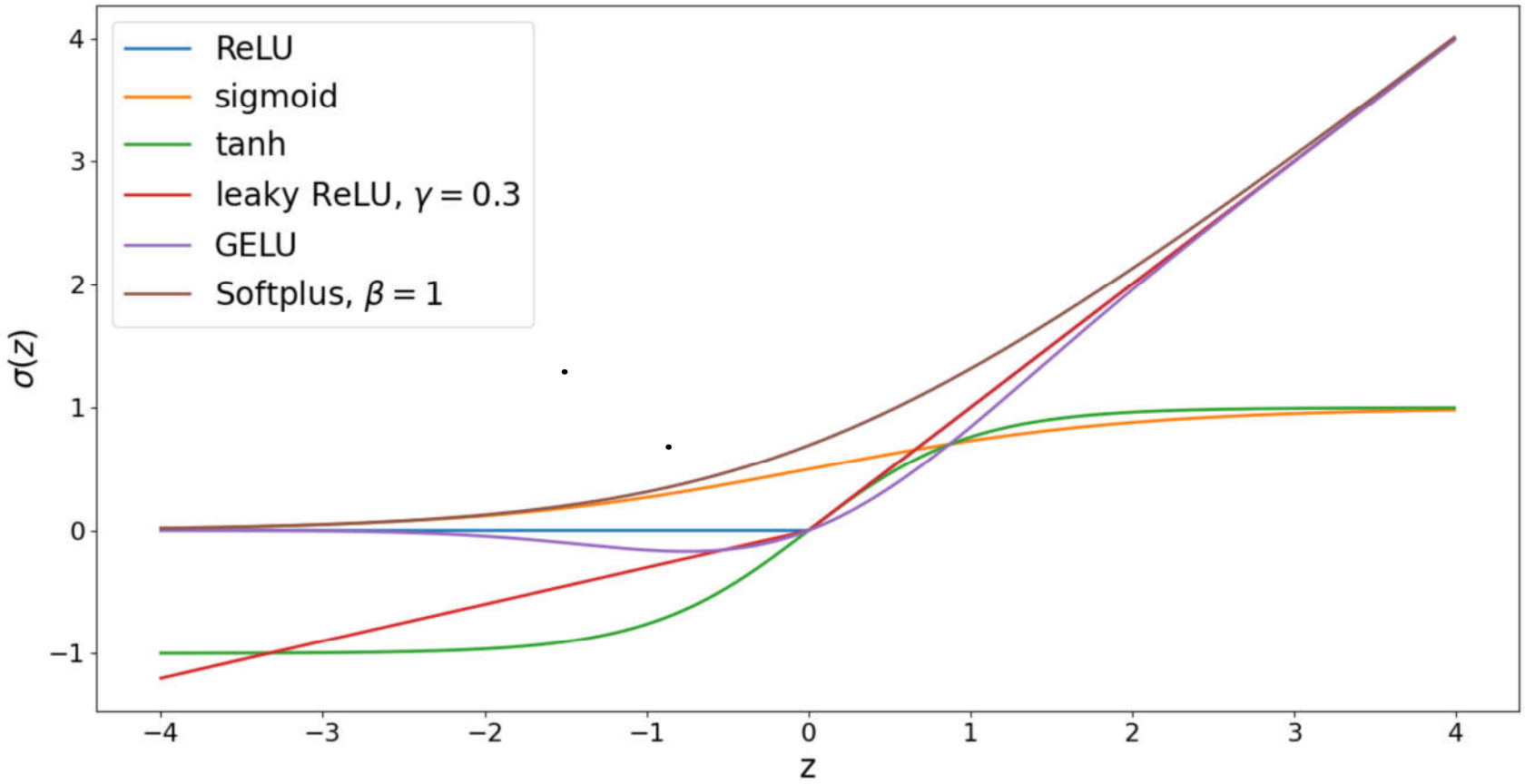
$$\sigma(z) = \max\{z, \gamma z\}, \gamma \in (0,1) \qquad \text{(leaky ReLU)}$$

$$\sigma(z) = \frac{z}{2}\left[1+\text{erf}(\frac{z}{\sqrt{2}})\right] \qquad \text{(GELU)}$$

$$\sigma(z) = \frac{1}{\beta}\log(1+\exp(\beta z)), \beta > 0 \qquad \text{(Softplus)}$$

Activation Functions





Why don't we use identity function?

$$
\begin{aligned}
\bar{h}_\theta(x) &= W^{[2]} a^{[1]} \\
&= W^{[2]} \sigma(z^{[1]}) \\
&= W^{[2]} z^{[1]} \\
&= W^{[2]} W^{[1]} x \\
&= \tilde{W} x
\end{aligned}
$$

$f_L(f_{L-1}(\cdots f_1(x)))$

$\circ_{i=1}^{L} f_i$

Connection to the kernel method…

$$\bar{h}_\theta(x) = W^{[r]}\phi_\beta(x) + b^{[r]}$$

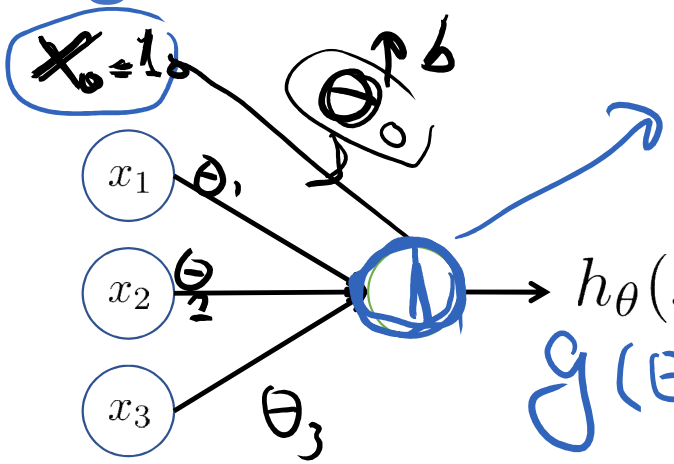
نقشه ویژگی

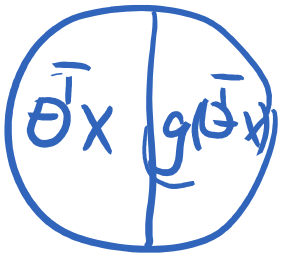
Neuron model: Logistic unit

$z = \theta^T x$

$a = \sigma(\theta^T x)$

$a = \frac{1}{1 + e^{-\theta^T x}}$





$b = \theta_0$

w, b

θ_i $i = 0 \ldots n$

$x_0 = 1$

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

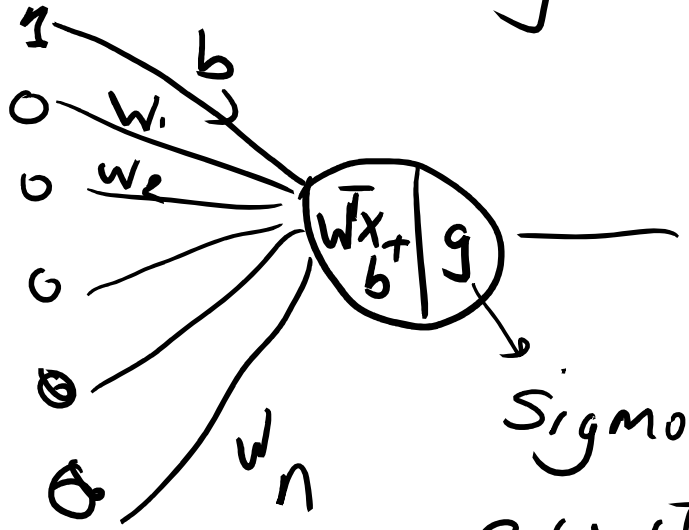
$g(\theta^T x)$

$\theta^T x = \theta_0 x_1 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$

$a = g(\theta^T x)$

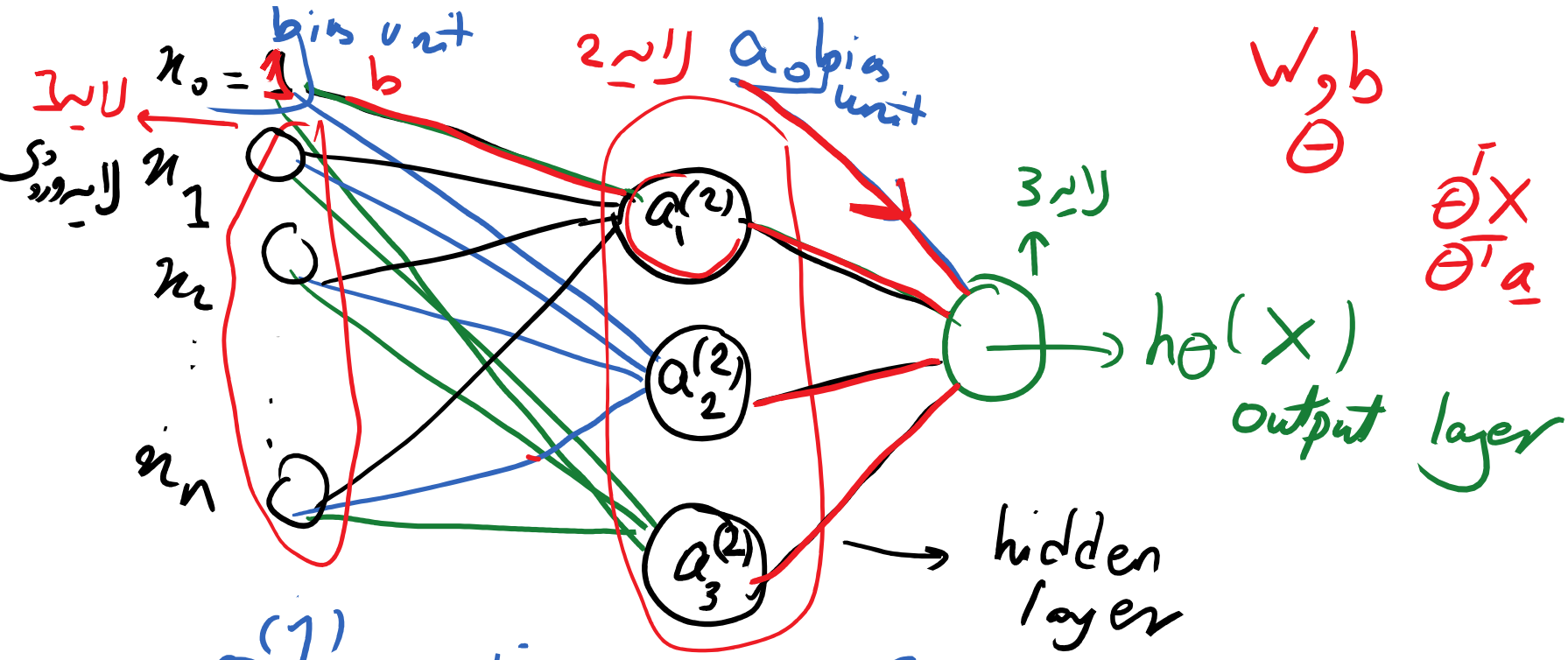
Sigmoid (logistic) activation function.

perceptron learning model:



$g(\bar{w}^T x + b)$

$$\begin{cases} \bar{w}^T x + b > 0 \rightarrow y = 1 \\ \bar{w}^T x + b < 0 \rightarrow y = 0 \end{cases}$$

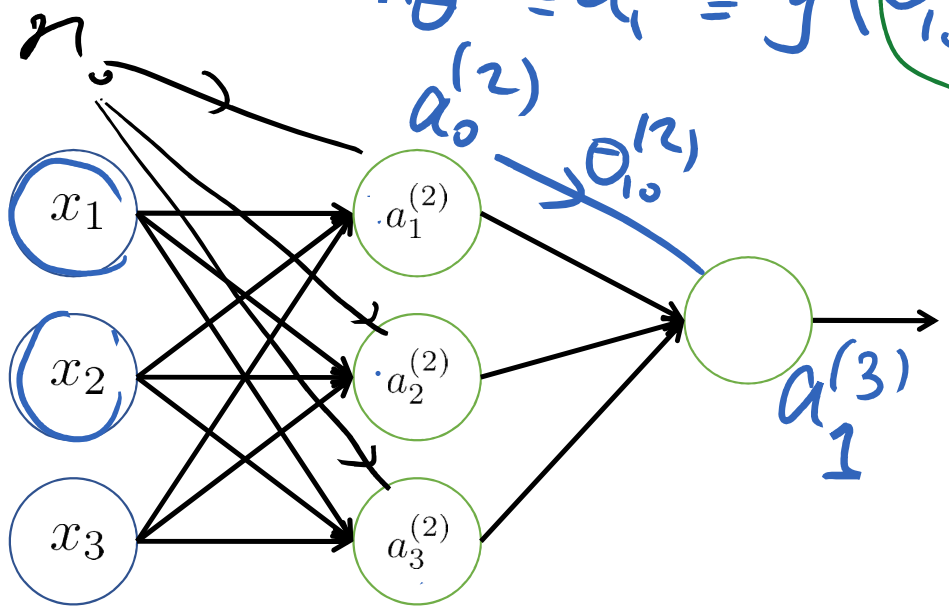


$\theta^{(1)}$: ماتریس وزن‌های لایه اول

$a_i^{(j)}$ = activation of unit i in layer j

$$a_1^{(2)} = g\left(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \dots \theta_{1n}^{(1)} x_n\right)$$

Neural Network



$a_i^{(l)}$ ناسین خروجی لایه i

$a^{(l)} = g(z^{(l)})$

$\Theta_{ij}^{(l)}$ شماره نورون خروجی — شماره نورون ورودی

$a_0^{(2)}$ $\Theta_{10}^{(2)}$ $a_1^{(3)}$

$h_\Theta(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$

$z_1^{(3)}$

$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$

$z_2^{(2)}$

$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$ → $z_3^{(2)}$

Activation function

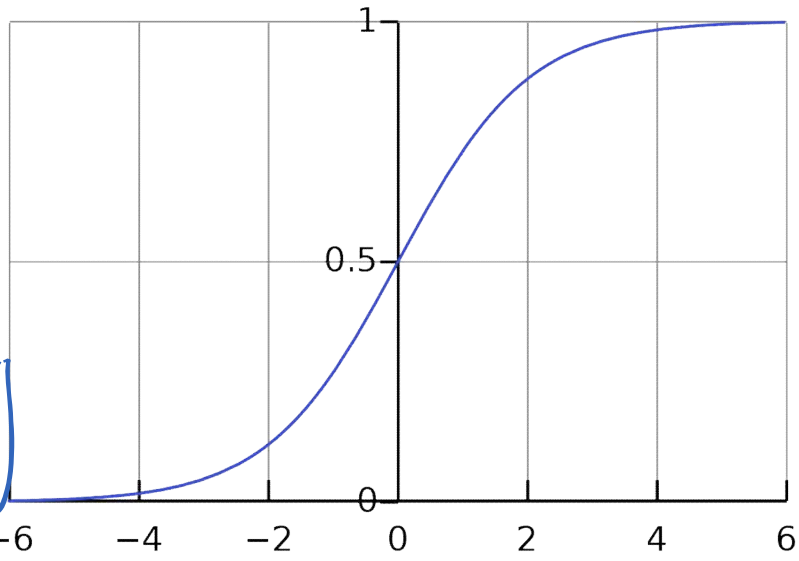
$a^{(l)} = g(w^T a^{(l-1)} + b^{(l)})$

$\Theta^T a^{(l-1)} \rightarrow \begin{bmatrix} 1 \rightarrow a_0^{(l-1)} \\ \vdots \end{bmatrix}$

Logistic, sigmoid

Function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Derivative:

$\sigma'(x) = \sigma(x)(1 - \sigma(x))$

$f(x)\big(1 - f(x)\big)$

range:

[0 , 1)

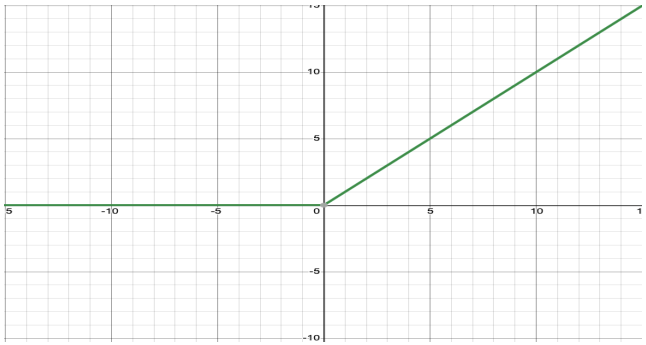
Activation function

ReLU

Function:

$$\begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$$

$$= \max\{0, x\} = x\mathbf{1}_{x>0}$$



$z = \theta^T x$

$a = g(z)$

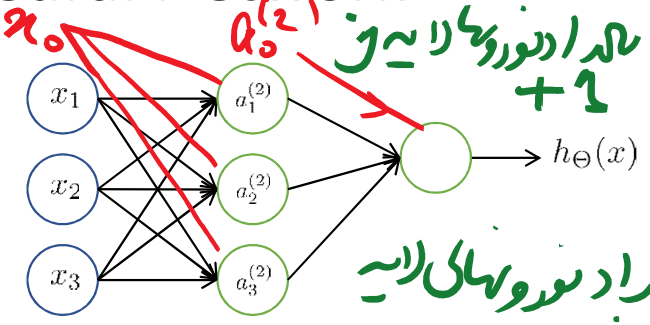
Derivative:

$$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$$

range:

$(0\ ,\ \infty)$

Neural Network



$a_i^{(j)}$ = "activation" of unit i in layer j

$\Theta^{(j)}$ = matrix of weights controlling function mapping from layer j to layer $j+1$

$$
\begin{cases}
a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3) \\
a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3) \\
a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)
\end{cases}
$$

$$
h_\Theta(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})
$$

If network has s_j units in layer j, s_{j+1} units in layer $j+1$, then $\Theta^{(j)}$ will be of dimension $s_{j+1} \times (s_j + 1)$

forward propagation

we can compute layer $l+1$'s activations $a^{(l+1)}$ as:

$$z^{(l+1)} = W^{(l)} a^{(l)} + b^{(l)}$$
$$a^{(l+1)} = g\left(z^{(l+1)}\right)$$

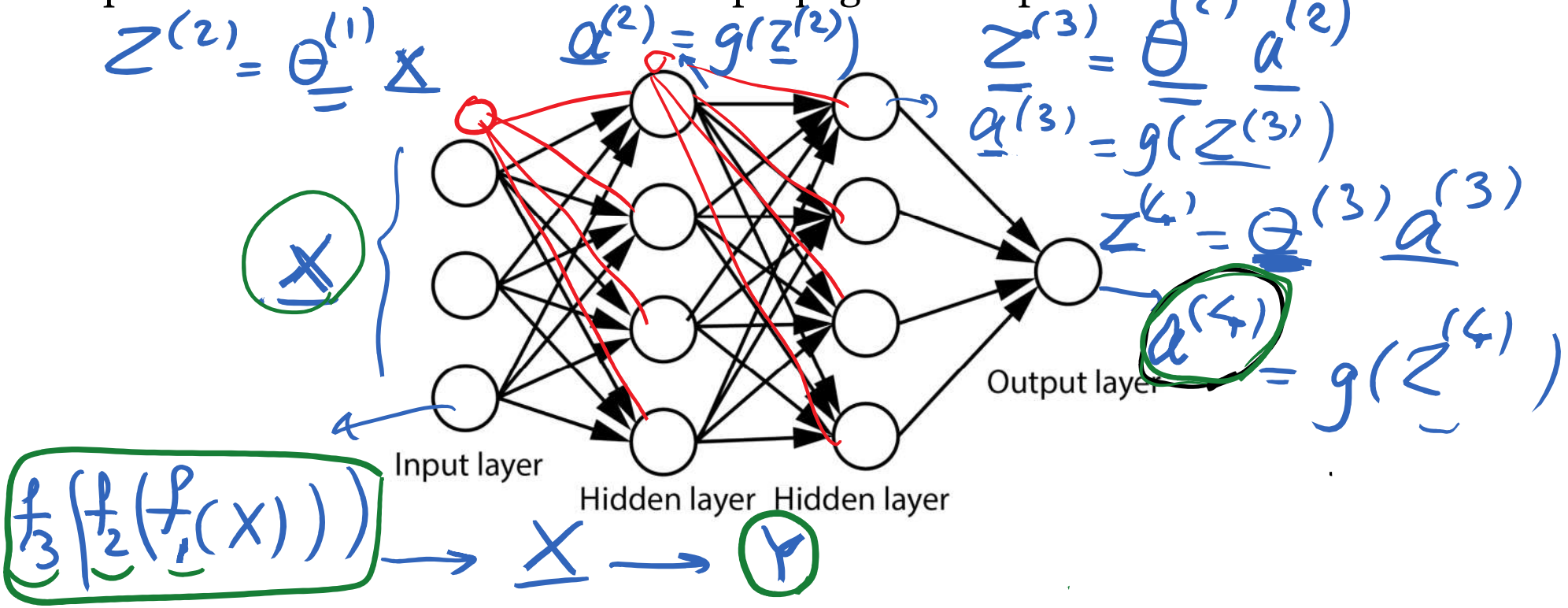
$$z^{(l+1)} = \begin{bmatrix} z_1^{(l+1)} & z_2^{(l+1)} & \cdots & z_{s_{l+1}}^{(l+1)} \end{bmatrix}^T$$

$s_{l+1} \times 1$

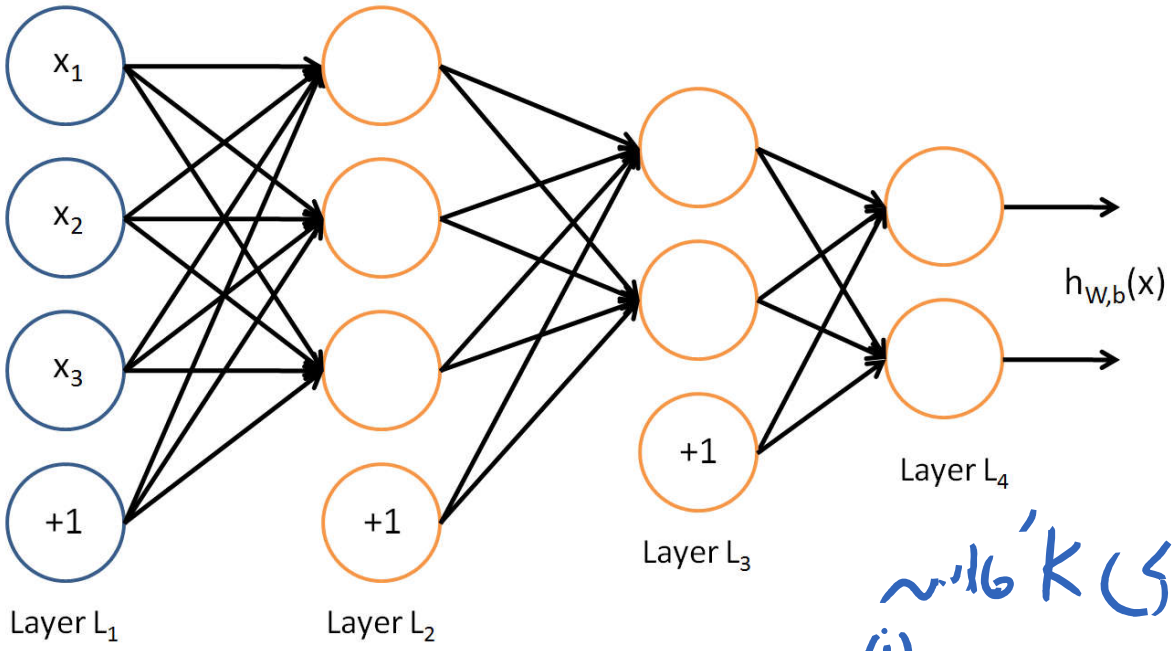
$z^{[L]}$: عدد الوحدات في الطبقة L $\longrightarrow$ $a^{[L]} = g(z^{[L]})$

Neural networks with multiple hidden layers

to compute the output of the network, we can successively compute all the activations in layer L_2, then layer L_3, and so on, up to layer L_{n_l}, using the equations that describe the forward propagation step.



Neural networks with multiple output



صفر و یک k کلاسه

$y^{(i)} = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ \vdots \\ 1 \end{bmatrix}$

$k = 4$

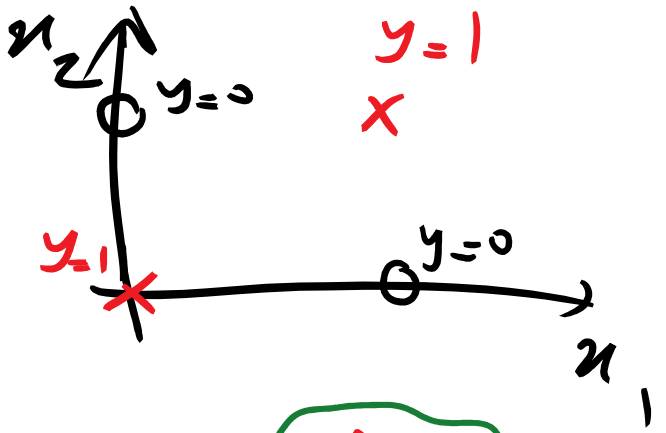
$y^{(i)} \in \mathbb{R}^4$

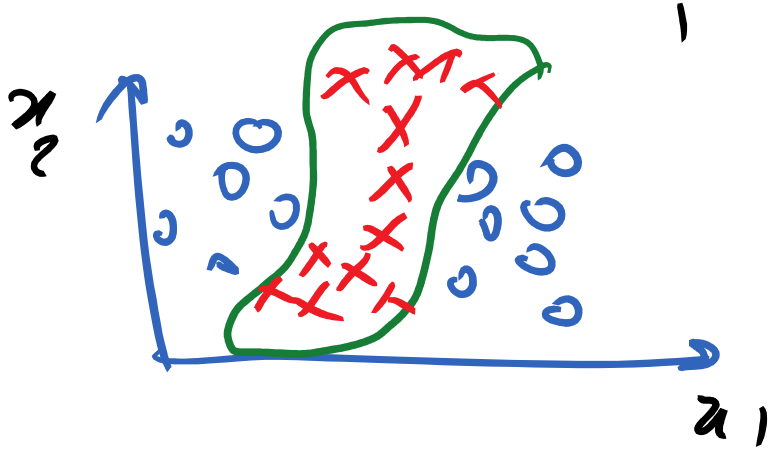
لایه آخر $a^{[L]} \in \mathbb{R}^4$

XOR / XNOR :

$y = x_1 \,(XNOR)\, x_2$

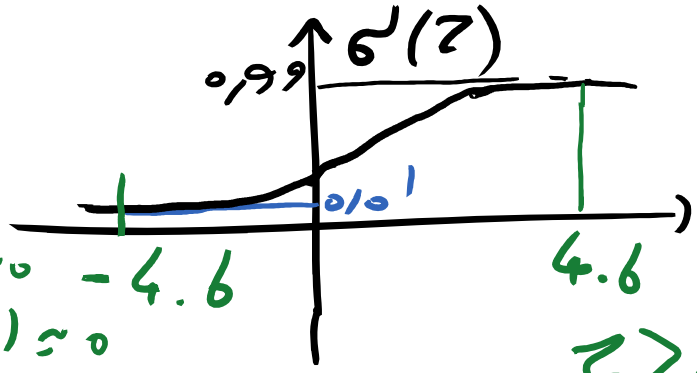
x_1	x_2	y
0	0	1
1	1	1
1	0	0
0	1	0





AND:

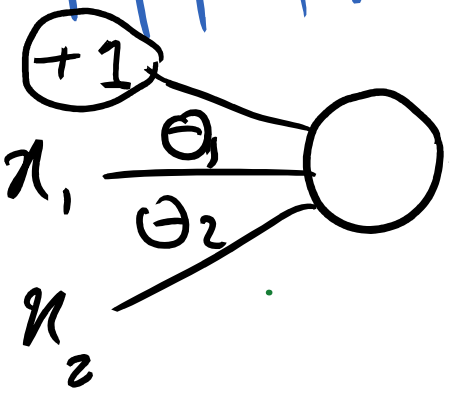
x_1	x_2	y	
0	0	0	$g(-30) \simeq 0$
0	1	0	$g(-10) \simeq 0$
1	0	0	$g(-10) \simeq 0$
1	1	1	$g(10) \simeq 1$



$$\frac{1}{1+e^{-z}}$$

$z > 4.6 \Rightarrow g(z) \simeq 1$

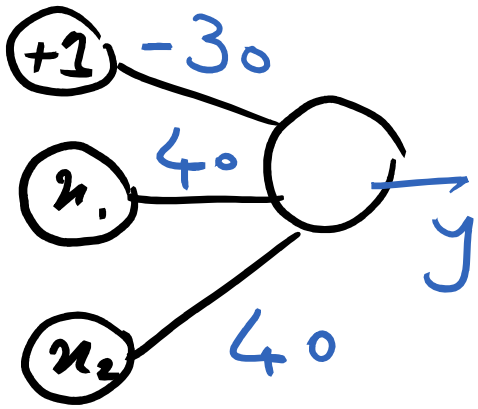
$z < -4.6 \Rightarrow g(z) \simeq 0$



$$y = h_\Theta(x) = g\left(\underset{-30}{\Theta_{10}^{(1)}} \times 1 + \underset{20}{\Theta_{11}^{(1)}} x_1 + \underset{20}{\Theta_{12}^{(1)}} x_2\right)$$

OR

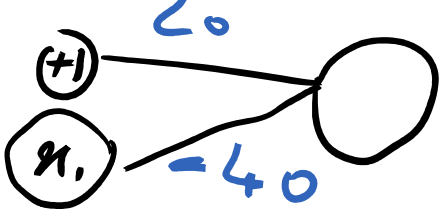
$y = x_1$ OR x_2



x_1	x_2	y	$h_\Theta(x)$
0	0	0	$g(-30) \simeq 0$
0	1	1	$g(+10) \simeq 1$
1	0	1	$g(10) \simeq 1$
1	1	1	$g(40) \simeq 1$

$= g(-30 + 40x_1 + 40x_2)$

$y = \text{Not}(x_1)$

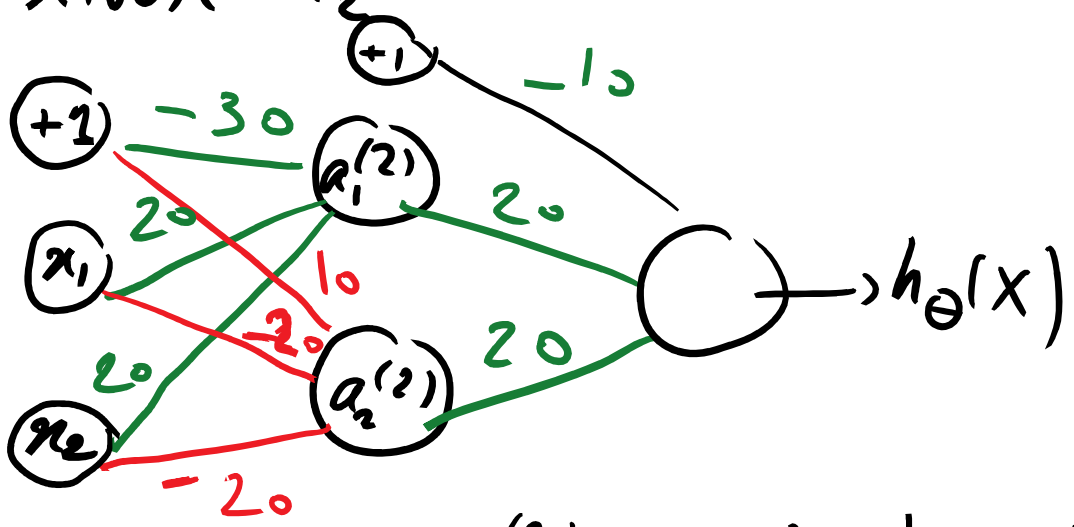


$y = h_\Theta(x)$

$g(20 - 40x_1)$

x_1	y	h_Θ
0	1	$g(20) \simeq 1$
1	0	$g(-20) \simeq 0$

2. XNOR x_1 x_2:



$$h_\Theta(x) = g(-10 + 20a_1^{(2)} + 20a_2^{(2)})$$

x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_\Theta(x)$
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

Backpropagation Algorithm

For the intermediate terms, we can use the chain rule to get:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_3} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{a}_4}\frac{\partial \boldsymbol{a}_4}{\partial \boldsymbol{\theta}_3}$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_2} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{a}_4}\frac{\partial \boldsymbol{a}_4}{\partial \boldsymbol{a}_3}\frac{\partial \boldsymbol{a}_3}{\partial \boldsymbol{\theta}_2}$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_1} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{a}_4}\frac{\partial \boldsymbol{a}_4}{\partial \boldsymbol{a}_3}\frac{\partial \boldsymbol{a}_3}{\partial \boldsymbol{a}_2}\frac{\partial \boldsymbol{a}_2}{\partial \boldsymbol{\theta}_1}$$

where each $\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_k} = \left(\nabla_{\boldsymbol{\theta}_k}\mathcal{L}\right)^\top$ is a d_k-dimensional gradient row vector, where d_k is the number of parameters in layer k.

Backpropagation Algorithm

Suppose we have a fixed training set $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(m)}, y^{(m)})\}$ of m training examples.

We can train our neural network using batch gradient descent. In detail, for a single training example (x, y).

we define the cost function with respect to that single example to be:

$$J(W, b; x, y) = \frac{1}{2} \left\| h_{W,b}(x) - y \right\|^2$$

Backpropagation Algorithm

This is a squared-error cost function. Given a training set of m examples, we then define the overall cost function to be:

$$J(W,b) = \left[\frac{1}{m}\sum_{i=1}^{m} J\left(W,b;x^{(i)},y^{(i)}\right)\right] + \frac{\lambda}{2}\sum_{l=1}^{n_l-1}\sum_{i=1}^{s_l}\sum_{j=1}^{s_{l+1}}\left(W_{ji}^{(l)}\right)^2$$

$$= \left[\frac{1}{m}\sum_{i=1}^{m}\left(\frac{1}{2}\left\|h_{W,b}\left(x^{(i)}\right)-y^{(i)}\right\|^2\right)\right] + \frac{\lambda}{2}\sum_{l=1}^{n_l-1}\sum_{i=1}^{s_l}\sum_{j=1}^{s_{l+1}}\left(W_{ji}^{(l)}\right)^2$$

W, b

L_2-norm

Backpropagation Algorithm

- Our goal is to minimize $J(W, b)$ as a function of W and b.
- To train our neural network
  - initialize each parameter $W_{ij}^{(l)}$ and each $b_i^{(l)}$ to a small random value near zero (say according to a $\text{Normal}(0, \epsilon^2)$ distribution for some small ϵ, say 0.01)
  - apply an optimization algorithm such as batch gradient descent.

Backpropagation Algorithm

One iteration of gradient descent updates the parameters W, b as follows:

$$W_{ij}^{(l)} = W_{ij}^{(l)} - \alpha \frac{\partial}{\partial W_{ij}^{(l)}} J(W, b)$$

$$b_i^{(l)} = b_i^{(l)} - \alpha \frac{\partial}{\partial b_i^{(l)}} J(W, b)$$

Backpropagation Algorithm

- how backpropagation can be used to compute:

$$\frac{\partial}{\partial W_{ij}^{(l)}} J(W, b; x, y)$$

$$\frac{\partial}{\partial b_i^{(l)}} J(W, b; x, y)$$

- the derivative of the overall cost function $J(W, b)$ can be computed as:

$$\frac{\partial}{\partial W_{ij}^{(l)}} J(W, b) = \left[\frac{1}{m} \sum_{i=1}^{m} \frac{\partial}{\partial W_{ij}^{(l)}} J\left(W, b; x^{(i)}, y^{(i)}\right) \right] + \lambda W_{ij}^{(l)}$$

$$\frac{\partial}{\partial b_i^{(l)}} J(W, b) = \frac{1}{m} \sum_{i=1}^{m} \frac{\partial}{\partial b_i^{(l)}} J\left(W, b; x^{(i)}, y^{(i)}\right)$$

Backpropagation Algorithm

In detail, here is the backpropagation algorithm:

1. Perform a feedforward pass, computing the activations for layers L_2, L_3, and so on up to the output layer L_{n_l}.

2. For each output unit i in layer n_l (the output layer), set

$$\delta_i^{(n_l)} = \frac{\partial}{\partial z_i^{(n_l)}} \frac{1}{2} \|y - h_{W,b}(x)\|^2 = -\left(y_i - a_i^{(n_l)}\right) \cdot g'\left(z_i^{(n_l)}\right)$$

$$a^{(n_l)} = y = g\left(z^{(n_l)}\right)$$

Backpropagation Algorithm

In detail, here is the backpropagation algorithm:

3. For $l = n_l - 1, n_l - 2, n_l - 3, \ldots, 2$

   For each node i in layer l, set

$$\delta_i^{(l)} = \left(\sum_{j=1}^{s_{l+1}} W_{ji}^{(l)} \delta_j^{(l+1)} \right) g'\left(z_i^{(l)}\right)$$

Backpropagation Algorithm

In detail, here is the backpropagation algorithm:

4. Compute the desired partial derivatives, which are given as:

$$\frac{\partial}{\partial W_{ij}^{(l)}} J(W, b; x, y) = a_j^{(l)} \delta_i^{(l+1)}$$

$$\frac{\partial}{\partial b_i^{(l)}} J(W, b; x, y) = \delta_i^{(l+1)}$$

Backpropagation Algorithm

re-write the algorithm using matrix-vectorial notation

We will use "•" to denote the element-wise product operator

1. Perform a feedforward pass, computing the activations for layers L_2, L_3, up to the output layer L_{n_l}, using the equations defining the forward propagation steps.

2. For the output layer (layer n_l), set

$$\delta^{(n_l)} = -\left(y - a^{(n_l)}\right) \bullet g'\left(z^{(n_l)}\right)$$

3. For $l = n_l - 1, n_l - 2, n_l - 3, \ldots, 2$, set

$$\delta^{(l)} = \left(\left(W^{(l)}\right)^T \delta^{(l+1)}\right) \bullet g'\left(z^{(l)}\right)$$

Backpropagation Algorithm

re-write the algorithm using matrix-vectorial notation

We will use "●" to denote the element-wise product operator

4. Compute the desired partial derivatives:

$$\nabla_{W^{(l)}} J(W, b; x, y) = \delta^{(l+1)} \left(a^{(l)}\right)^T$$
$$\nabla_{b^{(l)}} J(W, b; x, y) = \delta^{(l+1)}$$

Implementation note

In steps 2 and 3 above, we need to compute $g'\left(z_i^{(l)}\right)$ for each value of i.

Assuming sigmoid activation function

we would already have $a_i^{(l)}$ stored away from the forward pass through the network.

Thus, using the expression that we worked out earlier for $g'(z)$, we can compute this as $g'\left(z_i^{(l)}\right) = a_i^{(l)}\left(1 - a_i^{(l)}\right)$.

Implementation note

1. Set $\Delta W^{(l)} := 0, \Delta b^{(l)} := 0$ (matrix/vector of zeros) for all l.
2. For $i = 1$ to m,
3. Use backpropagation to compute $\nabla_{W^{(l)}} J(W, b; x, y)$ and $\nabla_{b^{(l)}} J(W, b; x, y)$.
4. Set $\Delta W^{(l)} := \Delta W^{(l)} + \nabla_{W^{(l)}} J(W, b; x, y)$.
5. Set $\Delta b^{(l)} := \Delta b^{(l)} + \nabla_{b^{(l)}} J(W, b; x, y)$.
6. Update the parameters:

$$W^{(l)} = W^{(l)} - \alpha \left[\left(\frac{1}{m} \Delta W^{(l)} \right) + \lambda W^{(l)} \right]$$

$$b^{(l)} = b^{(l)} - \alpha \left[\frac{1}{m} \Delta b^{(l)} \right]$$

$$\theta^{(l)}_{init} \sim N\left(0, \sqrt{\frac{2}{n^{(l)} + n^{(l-1)}}}\right)$$

Xavier / He initialization

$$\theta \sim Rand(-\epsilon, \epsilon)$$

Forward Pass:

$$z^{[1]} = w^{[1]} x + b^{[1]}$$

$$a^{[1]} = g(z^{[1]})$$

$$z^{[2]} = w^{[2]} a^{[1]} + b^{[2]}$$

$$a^{[2]} = g(z^{[2]})$$

$$\vdots$$

$$z^{[L]} = w^{[L]} a^{[L-1]} + b^{[L]}$$

$$a^{[L]} = g(z^{[L]})$$

θ w, b

$$z^{[L]} = \theta^{(L)T} a^{[L-1]}$$

$a_0^{(l)} = 1$ برای اضافه شدن بایاس لایه‌ها

$$J = l(a^{(L)}, y)$$

$$J = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log\left(h_\Theta(x^{(i)})_k\right) + (1-y_k^{(i)}) \log\left(1-h_\Theta(x^{(i)})_k\right) + \frac{\lambda}{2m}\sum_{l=1}^{L-1}\sum_{i=1}^{s_l}\sum_{j=1}^{s_{l+1}} (\Theta_{ij}^{(l)})^2$$

$a^{[L]}$

فقط تا اینجای Class entropy برای مسئله طبقه‌بندی k کلاسه

* برای k کلاسه $a^{[L]}$ هم k تا دارد.

k

همه چیز با $\frac{\partial J}{\partial z^{[L]}}$ شروع می‌شود و وارد روی آن $\frac{\partial J}{\partial \Theta^{[L]}}$ به دست می‌آید.

$$\frac{\partial J}{\partial \theta^{(l)}} = \frac{\partial J}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial \theta^{(l)}}$$

$\delta^{[l]}$ $\quad$ $a^{[l-1]T}$

← $\frac{\partial J}{\partial \theta}$ ~ $\frac{\partial J}{\partial z}$ (بردار)

Backprop.

$$\delta^{[L]} = \frac{\partial J}{\partial z^{[L]}} \rightarrow \delta^{[l-1]} = \frac{\partial J}{\partial z^{[l-1]}} = \frac{\partial J}{\partial z^{[L]}} \cdot \frac{\partial z^{[L]}}{\partial a^{[l-1]}} \cdot \frac{\partial a^{[l-1]}}{\partial z^{[l-1]}}$$

$\delta^{[L]}$ $\quad$ $\theta^{[l-1]}$ $\quad$ $g'(z^{[l-1]})$

$$\delta^{[l-1]} = (\theta^{[l-1]} \delta^{[L]}) .* g'(z^{[l-1]})$$

$$\delta^{[1]} = (\theta^{[1]} \delta^{[2]}) .* g'(z^{[1]})$$

الگوریتم بهینه سازی Backpropagation :

۱- Θ همه لایه ها مقداردهی اولیه شود

و حلقه به هنگام سازی Θ تشکیل شود.

$\Theta \sim Rand(-\epsilon, \epsilon)$ یا $\Theta^{(l)} \sim N(0, \sqrt{\frac{2}{n^{(l)} + n^{(l-1)}}})$

n_{in} و n_{out} دهر لایه برابر شوند با: $(S_l + 1) \times S_{l+1}$

۲- $\Delta^{(l)}_{ij} = 0$ ($\Delta^{(l)}$ ماتریس هم بعد $\Theta^{(l)}$ لایه l)

۳- for $i = 1 : m$ برای m داده train

$\{x^{(i)}, y^{(i)}\}_{i=1 \dots m}$

Forward → $a^{[1]}, z^{[2]}, a^{[2]} \rightarrow \dots \rightarrow z^{[L]}, a^{[L]}$

Backward → $\delta^{[L]} \dots \delta^{[1]}$

$\Delta^{(l)} := \Delta^{(l)} + \delta^{(l+1)} a^{(l)T}$ ← ماتریس هم بعد $\Theta^{(l)}$ لایه l

$D^{(l)} := \frac{1}{m} \Delta^{(l)} + \lambda \Theta^{(l)}$ → پایدارساز

بیانگر $\frac{\partial}{\partial \Theta}$

$D_{ij}^{(l)} = \frac{1}{m} \Delta_{ij}^{(l)}$

برای $i = 0$

چون برای $\theta_{i0}^{(l)}$ (بایاس)

هزینه که پایدار سازی ندارم

$$\Theta^{(l)} := \Theta^{(l)} - \alpha D^{(l)}$$

Learning Rate

end (for $i = 1 \dots m$ روی)

تکرار عددی برای

به هنگام سازی Θ ها

$(S_{l+1}) \times (S_{l+1}) : \Theta^{(l)}$ به

⑤ f نیز محدب است و ممکن است به $\min$ local برسد آیا